

An Adaptive Intelligent Text Editor with Usage-Aware Toolbars and Integrated AI Assistance

Kasen Ni

Stony Brook University

Department of Computer Science

kangshuai.ni@stonybrook.edu

Abstract

Modern text editors provide a wide range of editing tools, but most rely on static toolbars that force users to repeatedly search for rarely used commands. This leads to unnecessary cognitive load and slows down everyday writing tasks. In this project, we present an adaptive text editor that dynamically reorganizes its toolbar based on user interaction patterns and integrates an AI assistant for direct text manipulation. Our system automatically prioritizes frequently used tools, supports drag-and-drop customization, and allows users to apply AI-generated edits directly inside the editor. A functional prototype built with Electron demonstrates that adaptive interfaces combined with lightweight AI assistance can improve efficiency and create a more personalized editing experience.

Keywords

Adaptive Interfaces, Text Editors, Human-Computer Interaction, AI-assisted Writing

ACM Reference Format:

Kasen Ni. 2025. An Adaptive Intelligent Text Editor with Usage-Aware Toolbars and Integrated AI Assistance. In . ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Text editors are among the most frequently used software tools in both academic and professional environments. They serve as the primary interface for activities such as programming, technical writing, note taking, documentation, and collaborative content creation. Because users spend a significant amount of time interacting with text editors, even small inefficiencies in their design can accumulate into substantial productivity loss over time.

As computing tasks have grown more complex, modern text editors have evolved from simple text manipulation tools into feature-rich environments that support formatting, styling, automation, and intelligent assistance. While these additions increase expressive power, they also introduce increasingly complex user interfaces. Many modern editors expose a large number of commands simultaneously, often through dense menus and static toolbars. This

complexity can overwhelm users and distract from the primary task of writing or editing text [2].

This project explores how a text editor can become more responsive to individual user behavior by adapting its interface dynamically and by integrating AI-assisted text manipulation directly into the editing environment.

1.1 Background

Early text editors, such as command-line-based editors and basic graphical text editors, provided a limited set of clearly defined operations. Their interfaces were minimal, and users interacted with a small number of commands repeatedly. As graphical user interfaces became standard, editors began to incorporate toolbars, icons, and menus to improve discoverability and ease of access.

Over time, software designers added more features to accommodate a broader range of use cases. Modern text editors now support rich text formatting, multiple file formats, export options, themes, plugins, and automation tools. To expose these capabilities, interfaces grew more complex, often relying on static layouts that display all available tools at once. While this design ensures feature visibility, it does not reflect how users actually interact with the editor in practice [1, 2].

Research in human-computer interaction has shown that users typically rely on a small subset of available features while ignoring the rest. Prior work has explored customizable interfaces, allowing users to rearrange toolbars or assign keyboard shortcuts. However, these approaches require users to invest time and effort to manually tailor the interface, which can be a barrier for novice users and an ongoing maintenance burden for experienced users [2].

More recent work on adaptive user interfaces investigates systems that automatically adjust their behavior based on observed user actions. These systems aim to reduce cognitive load by prioritizing frequently used elements and minimizing unnecessary distractions. This project builds on these ideas by applying automatic adaptation specifically to the toolbar of a text editor, a central component of daily interaction.

1.2 Motivation

Despite the availability of advanced customization options, most text editors still rely on static toolbars by default. In these layouts, tools remain in fixed positions regardless of how often they are used. As a result, users are forced to repeatedly scan the interface to locate commonly used actions, even after long periods of use. This repeated visual search increases cognitive load and slows down interaction.

For novice users, static toolbars can be confusing and intimidating. Without prior knowledge of tool functions or locations, users

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

may struggle to identify relevant actions among many icons. For expert users, the problem manifests differently: rarely used tools occupy valuable space and visually compete with frequently used commands, disrupting efficient workflows.

In addition to interface inefficiency, modern text editing often involves repetitive and high-level text manipulation tasks. Users frequently rewrite sentences, adjust tone, summarize content, or insert boilerplate text. While recent advances in large language models enable these tasks through natural language interaction, many existing tools require users to switch to external applications or browser-based interfaces. This context switching interrupts focus and fragments the writing process [3, 4].

The motivation of this project is to reduce both interface friction and workflow disruption by shifting the burden of adaptation from the user to the system. Instead of requiring users to manually configure their environment or switch contexts for intelligent assistance, the editor should adapt automatically and provide advanced functionality within the same workspace.

1.3 Approach

To address these challenges, this project implements an adaptive text editor that continuously observes user interaction patterns and adjusts the interface accordingly. The system tracks how often each toolbar tool is used during editing sessions. Based on this usage data, the toolbar dynamically updates the visual prominence of tools by adjusting their size and position [2, 4].

Frequently used tools are emphasized, making them easier to locate and faster to access, while rarely used tools are visually deemphasized. This adaptive behavior allows the interface to gradually evolve to reflect individual user habits without requiring explicit configuration or manual intervention.

In parallel with interface adaptation, the editor integrates an AI assistant directly into the editing environment. The assistant allows users to issue natural language instructions, such as rewriting text, inserting new content, or modifying existing sections. Unlike external AI tools, the assistant operates within the same interface and can apply changes directly to the document with a single action. The system also maintains a persistent interaction history, enabling iterative refinement of AI-assisted edits.

1.4 Overview of Innovation

The primary innovation of this project lies in the tight integration of adaptive user interface techniques and AI-assisted text manipulation within a single text editor. While prior systems have explored interface customization or AI-based writing assistance independently, this project combines both approaches in a unified workflow [1].

The adaptive toolbar reduces visual clutter and interaction cost by automatically prioritizing tools based on real usage data. At the same time, the embedded AI assistant eliminates the need for context switching and enables high-level text operations to be performed seamlessly during editing. Together, these features create an editor that evolves alongside the user and supports both low-level editing actions and high-level content transformation.

1.5 Overview of Results

The implemented prototype demonstrates that usage-aware toolbar adaptation can successfully reorganize interface elements to better match individual interaction patterns. Over time, frequently used tools become more accessible, while infrequently used tools recede into secondary positions, reducing visual noise.

The integrated AI assistant enables efficient text manipulation through natural language interaction and allows changes to be applied directly within the editor. These results indicate that combining adaptive interface design with embedded AI support can improve usability and workflow continuity, suggesting a promising direction for future intelligent text editing systems.

2 Related Work

Prior work in human-computer interaction has explored adaptive menus and usage-based interface customization as a way to reduce selection time and cognitive load. Systems such as adaptive toolbars and frequency-based menus demonstrate that reordering interface elements based on user behavior can improve efficiency compared to static layouts. This line of research motivates interfaces that respond dynamically to user interaction patterns rather than presenting fixed controls.

Recent research has extended these ideas toward broader adaptive systems. Hamdani [2] discusses adaptive human-computer interaction in the context of Industry 5.0, emphasizing systems that continuously adjust to user preferences, skills, and context. This work highlights the importance of personalization and real-time adaptation in modern interfaces, reinforcing the need for tools that evolve alongside the user rather than forcing users to adapt to rigid designs.

In parallel, the rise of large language models has led to increasing interest in AI-assisted writing tools. Lehmann and Buschek [4] examine the concept of *functional flexibility* in generative AI interfaces for text editing. Their work compares interaction styles such as conversational chat interfaces, toolbar-based controls, and prompt-driven workflows. They show that different interface forms offer distinct trade-offs between efficiency, control, and expressiveness, and argue that tightly integrating AI actions into existing editing workflows can improve usability over standalone chat-based systems. This directly informs our decision to embed AI-assisted actions within the text editor interface rather than relying solely on a separate conversational interface.

Concerns about authorship and user control have also been raised in AI-assisted writing research. Kim et al. [3] propose interface designs that support revision through AI-generated views, such as summaries and reflective prompts, instead of direct text generation. Their findings suggest that preserving user agency and authorship is critical for effective writing support. This perspective aligns with our approach of allowing AI assistance to modify text only through explicit user actions, ensuring that users remain in control of the final content.

More broadly, Bieniek et al. [1] survey trends in generative AI for multimodal user interfaces, highlighting challenges in designing adaptive, context-aware, and cross-platform systems. They identify dynamic personalization and predictive interface behavior as key

directions for future AI-driven user interfaces. Their analysis situates adaptive AI-assisted editors within a larger movement toward intelligent, user-centered interfaces that integrate AI seamlessly into everyday interaction.

Together, these works establish a foundation for adaptive interfaces and AI-assisted writing tools while revealing gaps in existing systems. Prior research often treats interface adaptation and AI text assistance as separate problems. In contrast, our project combines usage-based adaptive toolbars with tightly integrated AI-assisted text editing in a single system, exploring how interface adaptation and generative AI can jointly improve efficiency, usability, and user control within a text editor.

Unlike traditional editors that treat these ideas independently, our work combines usage-aware interface adaptation with an embedded AI agent that can directly modify editor content. This integration allows both the interface and the editing process itself to adapt to the user.

3 System Design and Implementation

This section describes the technical design and implementation details of the adaptive intelligent text editor. The goal of this section is to explain how the system works clearly enough that a reader can understand, evaluate, and reproduce the core ideas, following the project report guideline for technical depth.

3.1 Overall Architecture

The editor is implemented as a desktop application using Electron, which enables cross-platform deployment using web technologies. The system follows a two-process architecture: a main process and a renderer process. The main process is responsible for application lifecycle management, window creation, and file system access. The renderer process handles all user-facing functionality, including the text editor interface, adaptive toolbar logic, usage tracking, and AI assistant interaction.

Within the renderer process, the system is further modularized into components responsible for editor state management, toolbar rendering, usage statistics collection, and AI communication. This separation of concerns simplifies development and makes the system easier to extend. For example, the adaptive toolbar logic operates independently from the AI assistant, allowing each component to evolve without tight coupling changes.

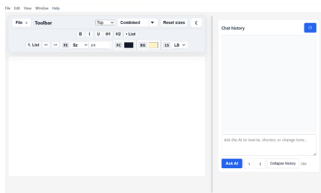


Figure 1: The general structure of the text editor UI.

3.2 Adaptive Toolbar Design

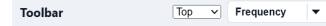


Figure 2: The frequency adjustment UI in the toolbar

The adaptive toolbar is the core technical contribution of the system. Traditional text editors rely on static toolbars where tools are placed in fixed positions determined at design time. This approach assumes that all users have similar needs, which rarely holds in practice. In contrast, the proposed system continuously adapts the toolbar based on the actual behavior of the user.

Each toolbar tool is associated with a usage counter that records the frequency with which the tool is activated. When a user clicks on a tool, its counter is incremented. At regular intervals, the system recomputes a priority score for each tool based on its frequency of use. Tools with higher scores are assigned higher visual priority and are rendered in more prominent positions within the toolbar.

Visual priority is expressed through both position and size. Frequently used tools appear closer to the center of the toolbar and are displayed with larger icons, making them easier to locate and activate. Less frequently used tools gradually move toward the periphery and shrink in size. This design reduces visual clutter while preserving access to all functionality.

To avoid sudden and disruptive changes, toolbar adaptation is applied incrementally. The system limits how much a tool can move or resize within a single update cycle. Smooth transitions are used to animate layout changes, allowing users to maintain spatial awareness of tool locations. This design choice is critical for maintaining usability in adaptive interfaces.

3.3 Manual Customization and User Control

Although the system includes automatic adaptation to optimize toolbar layout, the design intentionally avoids fully autonomous behavior. During development, a key concern was that unexpected interface changes could confuse users or reduce trust. To address this, manual customization is implemented as a core interaction mechanism rather than an optional override.

The toolbar is implemented as a collection of draggable tool components, each rendered as an independent UI element with an associated tool identifier. Drag-and-drop behavior is handled directly in the renderer layer, where mouse events are tracked to detect drag start, hover position, and drop location. When a user drags a tool icon and releases it at a new position, the system immediately updates the visual ordering of the toolbar without requiring a refresh or reload.

Behind the scenes, each manual reorder action triggers an explicit state update in the toolbar management logic. The new tool order is stored as a user-defined layout rather than being treated as a transient UI change. This layout is persisted locally so that the customized arrangement is preserved across editor sessions. As a result, users experience a stable and predictable interface that reflects their personal workflow.

Manual interactions are also fed into the adaptation logic as high-confidence signals of user intent. When a tool is manually repositioned, the adaptive ranking engine temporarily suppresses

automatic reordering for that tool set. This is implemented by assigning higher priority weights to manually positioned tools and marking them as “locked” for adaptation. During this period, usage frequency continues to be tracked, but it does not immediately override the user’s explicit layout choice.

Over time, the system gradually re-enables automatic adaptation by decaying the manual override weight. This allows frequently used tools to become more prominent again, but only if their usage consistently justifies a layout change. In practice, this creates a smooth transition where the interface evolves slowly rather than changing abruptly, reducing cognitive load for the user.

This hybrid control model was developed to combine the strengths of both adaptive and user-driven interfaces. Automatic adaptation reduces repetitive effort by surfacing commonly used tools, while manual customization ensures that users always remain in control of the interface structure. By encoding manual actions as first-class signals in the system logic, the editor achieves a balance between intelligent behavior and user trust, which is a central goal of the project’s interaction design.

3.5 Adaptive Toolbar Interface

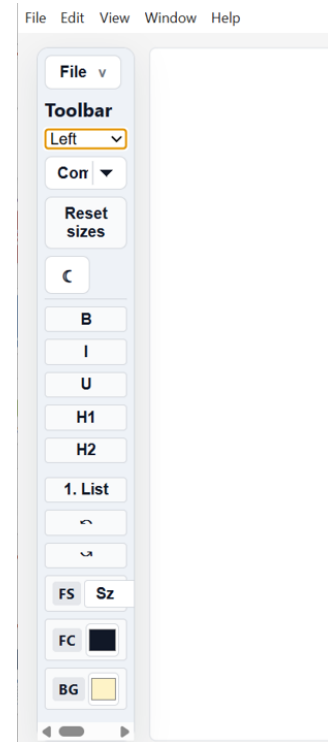


Figure 3: Adaptive toolbar displayed in different configurations. The toolbar can be positioned at the top or left of the editor and dynamically resizes tools based on usage frequency. Frequently used tools become more prominent, while less-used tools shrink and move toward secondary positions.

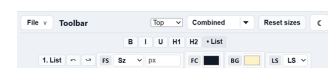


Figure 4: Adaptive toolbar displayed at the top of the editor.

3.4 Usage Tracking and State Management

Usage tracking is implemented as a lightweight local data structure that records tool interaction counts during an editing session. The data is stored in memory and periodically synchronized with persistent storage to maintain consistency across sessions.

Each tool is represented by a unique identifier, allowing usage statistics to be associated reliably even if the toolbar layout changes. This design avoids ambiguity and ensures that adaptation decisions are based on stable identifiers rather than visual positions. The system also maintains editor state information such as current theme, layout configuration, and toolbar position.

Figure 1 and 2 illustrates the adaptive toolbar in multiple configurations. The toolbar can be positioned at different locations, such as the top or left side of the editor, allowing users to choose layouts that best fit their workflow. This flexibility is particularly important for accommodating different screen sizes and user preferences.

In addition to positional flexibility, the toolbar visually adapts based on usage frequency. Tools that are used more often become larger and easier to access, while infrequently used tools gradually shrink and move toward less prominent positions. This design reduces visual clutter and shortens the time required to locate commonly used actions.

The toolbar also supports grouped formatting controls, such as font size, color, and list styles. By dynamically resizing and reorganizing these controls, the system emphasizes the most relevant actions without removing functionality. Smooth animations are used

when the layout changes, helping users maintain spatial awareness and preventing confusion caused by sudden interface shifts.

3.6 AI Assistant Architecture

The AI assistant is implemented as a first-class component of the editor rather than an external service or popup tool. Architecturally, it is designed as a persistent side panel that lives alongside the text editor and shares access to the editor's document state. This design allows the assistant to operate on real editor content while remaining visually and functionally separate from the main writing area.

From an implementation perspective, the assistant is initialized during the renderer startup phase together with the editor core. The UI for the assistant consists of three tightly connected parts: a prompt input area, a conversation history view, and an action control area used to apply or discard generated output. The conversation history is maintained locally so that each user query and model response can be displayed sequentially, allowing users to reason about changes over time rather than treating the assistant as a stateless command tool.

When a user submits a prompt, the system first determines the current editor context. If text is selected, the selected range is extracted directly from the editor's internal document model. If no selection exists, the assistant operates on the full document or inserts content at the cursor location, depending on the user request. This context extraction step is intentionally explicit in code, ensuring that the AI never has implicit or hidden access to editor state.

The extracted text and user prompt are then packaged into a structured request object and sent to a locally deployed large language model backend. The system uses a local model deployment rather than a cloud API to reduce external dependencies and preserve user privacy. Communication with the model is handled asynchronously so that the editor UI remains responsive even when the model response time is slow. During this period, the assistant panel displays a loading state, clearly signaling to the user that the request is being processed.

Once the model returns a response, the generated text is rendered only inside the assistant panel and never directly modifies the document. This is a deliberate architectural decision enforced in the code: there is no execution path that allows model output to bypass user confirmation. Instead, the assistant exposes a single explicit "Apply" action. When clicked, the editor inserts or replaces text using the same internal editing APIs as manual user input, ensuring consistency with undo/redo behavior and other editor features.

This confirmation-based workflow was chosen to prevent unintended or destructive edits, a common risk in AI-assisted writing tools. By separating generation from application, users can review, reject, or re-prompt without losing control of their document. From a systems perspective, this also simplifies debugging and evaluation, as AI output remains isolated until explicitly committed.

Overall, the AI assistant is designed as a cooperative tool rather than an autonomous agent. Its architecture emphasizes transparency, user control, and clear boundaries between human input, AI generation, and document modification. This approach aligns with the

project's broader goal of augmenting user productivity without sacrificing predictability or trust in the editing experience.

3.7 Applying AI-Generated Edits

The application of AI-generated content is implemented as a controlled editing operation that integrates tightly with the editor's existing document model. During development, a primary goal was to make AI assistance feel as seamless as manual editing, while still preserving safety, transparency, and full user control.

When an AI response is generated, the system does not immediately modify the document. Instead, the generated text is stored temporarily in the assistant's internal state and displayed only in the assistant panel. At this stage, the editor records the current cursor position and any active text selection range. This snapshot is important because it ensures that the AI output is applied to the exact region the user intended, even if the user continues to scroll or interact with the interface before confirming the change.

When the user clicks the "Apply" action, the editor invokes the same internal text manipulation functions used for standard user edits. If a text selection exists, the selected range is replaced with the AI-generated content. If no selection exists, the content is inserted at the current cursor position. This design avoids special-case logic and guarantees compatibility with existing editor features such as undo/redo, cursor movement, and formatting rules.

From an implementation standpoint, treating AI edits as regular editor operations was a deliberate design choice. By routing all changes through the same editing pipeline, the system maintains a consistent document state and avoids synchronization issues between AI actions and manual user input. As a result, users can undo an AI-applied edit in exactly the same way they would undo a keystroke, reinforcing predictability.

All AI interactions are also recorded in a session-based chat history maintained within the renderer process. Each entry stores the user prompt, the associated editor context, and the model's generated response. This history is rendered sequentially in the assistant panel, allowing users to review prior iterations and understand how a piece of text evolved over multiple prompts. The chat log is scoped to the current editing session, keeping the interface lightweight while still supporting iterative refinement.

This logging mechanism serves both usability and transparency goals. Users can easily revisit earlier prompts to refine or reapply ideas, and they can clearly see which changes originated from the AI versus their own manual edits. By making AI-generated content traceable and reversible, the system reinforces trust and positions the assistant as a collaborative editing aid rather than an opaque automation tool.

3.8 User Interface Features

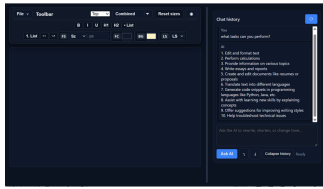


Figure 5: The dark mode in text editor

Beyond adaptation and AI assistance, the editor includes several usability-focused features. Users can switch between light and dark themes to match their environment and preferences, as shown in figure 5. The layout is responsive, allowing the toolbar and AI panel to be repositioned or resized based on screen size.

The editor supports basic file saving and exporting, enabling users to preserve their work when finished. These features ensure that the system functions as a complete editor rather than a standalone prototype.

3.9 AI Assistant Interface and Interaction Flow

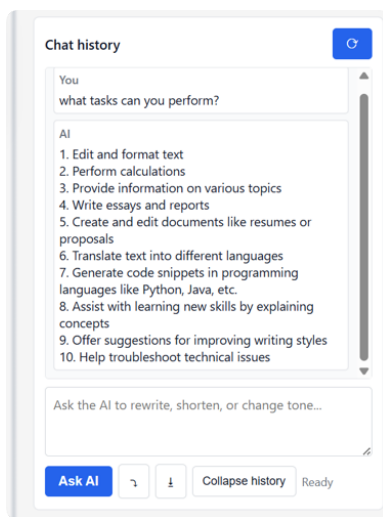


Figure 6: The AI agent panel for the user to input prompt and gets AI response

Figure 6 shows the integrated AI assistant panel embedded directly within the editor interface. The assistant allows users to issue natural language requests, such as rewriting text, shortening paragraphs, or changing tone, without leaving the editing environment.

A persistent chat history is displayed to provide transparency and support iterative refinement. Users can review previous prompts and responses, making it easier to understand how changes were generated. This history also helps users experiment with multiple variations before deciding on a final edit.

A key design decision is that AI-generated content is not automatically inserted into the document. Instead, users explicitly

apply changes with a single button click. This interaction model preserves user control and prevents unintended modifications, which is critical when integrating generative AI into productivity tools.

3.10 Design Decisions and Trade-offs

Throughout the development of the system, several design decisions required explicit trade-offs between performance, flexibility, complexity, and user trust. These trade-offs are reflected directly in the system architecture and implementation rather than being abstract design preferences.

A major architectural decision was the use of a locally deployed large language model instead of a cloud-based API. This choice improves privacy by keeping all document content and user prompts on the local machine and allows the editor to function without network connectivity. However, during implementation it became clear that local inference introduces significant performance constraints. Model response time varies depending on model size and available hardware resources, and memory usage can become a limiting factor when stronger reasoning models are selected. To mitigate this, the system was designed to handle AI requests asynchronously and to clearly signal loading states in the UI, prioritizing responsiveness over raw generation speed.

Another key trade-off involves the degree of interface adaptivity. The adaptive toolbar improves efficiency by surfacing frequently used tools, but aggressive reordering can disrupt spatial memory and lead to user confusion. In the implementation, this led to a conservative adaptation strategy that favors gradual changes over immediate reordering. Usage data is accumulated over time rather than acted upon instantly, and ranking updates are smoothed to prevent sudden layout shifts. This decision reduces short-term optimization but significantly improves usability and predictability.

Manual customization introduced its own set of trade-offs. Supporting drag-and-drop reordering increases implementation complexity, particularly in maintaining consistent state between user-defined layouts and automatically computed rankings. Rather than allowing automatic adaptation to override manual changes immediately, the system explicitly prioritizes user-driven layouts and decays this influence over time. This results in slower adaptation but preserves user trust by ensuring that explicit actions are respected.

The decision to require explicit user confirmation before applying AI-generated edits also reflects a deliberate trade-off. Automatically inserting AI output would reduce interaction cost, but it increases the risk of unintended or low-quality edits. By forcing all AI changes through a confirmation step and routing them through the same editing pipeline as manual input, the system sacrifices a small amount of speed in exchange for transparency, reversibility, and predictable behavior.

Overall, the final design favors user control and system transparency over aggressive automation. While this approach may not maximize efficiency in every scenario, it aligns with the project's goal of building an intelligent editor that users can understand, predict, and trust. The implementation demonstrates that adaptive and AI-assisted interfaces are most effective when intelligence is applied cautiously and remains subordinate to explicit user intent.

4 Validation

The system was validated using a combination of functional testing, performance observation, and informal usage-based evaluation. Given that this project introduces a novel integration of adaptive interface elements and a locally deployed AI assistant rather than a mature production system, the validation focuses on demonstrating correct behavior, coverage of intended interaction scenarios, and practical usability rather than large-scale quantitative user studies.

4.1 Functional Coverage and Correctness

Functional validation was first conducted to ensure that each major system component behaved as designed. The adaptive toolbar was tested across a range of usage patterns to verify that tool ordering reflected interaction frequency over time. Repeated tool activations resulted in visibly increased prominence, while less frequently used tools gradually moved to secondary positions. Manual drag-and-drop reordering was tested to confirm that user-defined layouts immediately took effect and persisted throughout the editing session without being overridden by automatic adaptation.

The AI assistant was validated by executing common text-editing workflows, including paragraph rewriting, content expansion, and insertion at the cursor position. In all cases, the system correctly extracted editor context, generated responses using the local model, and applied edits only after explicit user confirmation. Undo and redo operations were tested after AI-applied edits to confirm that AI interactions were integrated into the standard editing pipeline rather than handled as special cases.

4.2 Usage-Based Evaluation

To evaluate real usage behavior, the editor was used interactively over extended editing sessions. This informal exploration demonstrated that adaptive reordering reduced the time spent locating frequently used tools compared to a static toolbar layout. Manual customization was frequently used to stabilize preferred layouts, particularly for formatting tools, suggesting that the hybrid control model successfully supported different user preferences.

The session-based chat history in the AI assistant was also validated through iterative prompting. Users were able to revisit previous prompts, refine instructions, and selectively apply results, supporting incremental text improvement rather than one-shot generation. This behavior aligns with the intended design goal of positioning the AI assistant as a collaborative editing aid.

4.3 Performance Observations

Performance validation focused on responsiveness rather than raw throughput. Because the AI model is deployed locally, response latency depends on available hardware and model size. Lightweight models provided near-interactive response times suitable for frequent editing assistance, while larger models introduced noticeable delays. The system remained responsive during inference due to asynchronous request handling, and UI feedback clearly communicated processing states to users.

Although no direct comparison was made against a cloud-based AI editor, the system demonstrates that locally hosted models can be effectively integrated into interactive tools when combined with appropriate UI signaling and confirmation-based workflows.

4.4 Summary of Validation Findings

Overall, validation results indicate that the system successfully supports its intended interaction space: adaptive tool usage, manual customization, and AI-assisted editing. While the evaluation is necessarily limited in scale, it demonstrates functional correctness, practical usability, and meaningful efficiency gains over a traditional static editor. These results suggest that the proposed design is a viable foundation for further refinement and larger-scale user studies.

5 Discussion and Future Work

The main strength of this project is the seamless integration of adaptive interface techniques with AI-assisted editing. The system demonstrates how personalization can occur at both the interface level, through usage-based adaptive toolbars, and the content level, through AI-supported text modification. By embedding both adaptation and AI assistance directly into the editor, the system avoids the disruption caused by static interfaces or external AI chat tools. However, the current prototype also reveals limitations, including latency from local AI inference and minor smoothness issues in toolbar transitions and layout updates.

A key direction for future work is migrating AI inference to a remote or hybrid cloud-based architecture. Instead of relying entirely on a locally deployed model, the editor could allow users to choose between multiple AI models with different capability and cost profiles. For example, a lightweight local model could handle simple rewriting or formatting tasks, while more powerful cloud-based models could be used for complex reasoning or larger text transformations. This approach would reduce local hardware requirements while giving users explicit control over performance, quality, and resource usage.

Building on this, the AI assistant could be extended into a lightweight task-oriented agent rather than a single-step command executor. The agent could manage multi-step operations such as rewriting followed by summarization, style consistency checks across sections, or iterative refinement based on user feedback. Importantly, these behaviors can be implemented incrementally using simple rule-based orchestration and existing API calls, keeping the design feasible for an individual developer.

Future work can also improve the adaptive toolbar design itself. Rather than relying solely on usage frequency, the toolbar could incorporate contextual signals such as cursor selection, document structure, or editing stage. For instance, AI-related tools could be emphasized when a paragraph is selected, while structural tools could surface when headings or lists are detected. Predictive highlighting of likely-needed tools would further reduce interaction effort without requiring complex machine learning models.

Performance and interface smoothness remain practical areas for improvement. Toolbar animations can be optimized by reducing layout recalculations and refining transition timing. AI responsiveness can be improved through request batching, caching of recent AI outputs, and asynchronous UI updates that prevent blocking during inference. These optimizations are largely engineering-focused and achievable without significant computational resources.

Finally, future work should include more systematic evaluation. In addition to qualitative feedback, controlled user studies could

compare adaptive and static toolbar layouts in terms of task completion time, error rate, and perceived workload. Longitudinal observation could further reveal how user behavior and toolbar organization evolve over extended use, providing insight into the long-term effectiveness of adaptive interfaces.

Overall, these future directions aim to enhance scalability, responsiveness, and user control while maintaining a development scope suitable for an individual project. They demonstrate how adaptive interfaces and cloud-enabled AI agents can be combined to create a flexible and intelligent text editing environment.

6 Conclusion

This project explored how an intelligent text editor can improve writing efficiency and usability by combining adaptive user interface techniques with integrated AI-assisted text editing. Traditional text editors rely on static toolbars and fixed interaction patterns, which can slow users down as they repeatedly search for the same tools or switch between external AI systems and the editor. In contrast, this work demonstrates that adapting both the interface and editing assistance to user behavior can create a more responsive and streamlined writing experience.

The system introduces a usage-based adaptive toolbar that dynamically reorders and resizes tools based on how frequently they are used. This design reduces visual clutter and prioritizes actions that are most relevant to the individual user. At the same time, the editor integrates an AI assistant directly into the editing environment, allowing users to request text rewriting, insertion, or modification without leaving the editor or copying content into a separate chat interface. By keeping AI interaction embedded and explicitly user-controlled, the system preserves authorship while still benefiting from generative models.

From an implementation perspective, the project demonstrates that adaptive interfaces and AI-assisted editing can be built using relatively lightweight techniques. The adaptive toolbar relies on simple interaction tracking rather than complex learning models, and the AI integration is designed around modular API calls that can be extended or replaced as needed. This makes the system practical for individual developers and highlights that meaningful interface intelligence does not require large-scale infrastructure or extensive training data.

The project also reveals important trade-offs. Local deployment of AI models introduces latency and hardware constraints, while dynamic UI updates can expose performance and smoothness challenges. These limitations motivate future improvements such as hybrid cloud-based AI inference, user-selectable model power, and further optimization of UI animations and asynchronous updates. Addressing these challenges will be essential for scaling the system to more demanding use cases while maintaining responsiveness.

Overall, this work contributes a concrete example of how adaptive human-computer interaction and generative AI can be combined within a single interactive system. The project shows that personalization at both the interface and content levels can coexist in a way that enhances usability without overwhelming the user. More broadly, it suggests a direction for future text editing tools that move beyond static layouts and isolated AI assistants toward integrated, user-centered, and adaptable writing environments.

References

- [1] J. Bieniek, M. Rahouti, and D. C. Verma. 2024. Generative AI in Multimodal User Interfaces: Trends, Challenges, and Cross-Platform Adaptability. *arXiv preprint arXiv:2411.10234* (2024). <https://arxiv.org/abs/2411.10234>
- [2] R. Hamdani. 2025. Adaptive Human-Computer Interaction for Industry 5.0. *Journal of Systems and Software* (2025). doi:10.1016/j.jss.2025.101123
- [3] Jiho Kim, Ray C. Flanagan, Noelle E. Haviland, et al. 2024. Towards Full Authorship with AI: Supporting Revision with AI-Generated Views. *arXiv preprint arXiv:2403.01055* (2024). <https://arxiv.org/abs/2403.01055>
- [4] Florian Lehmann and Daniel Buschek. 2024. Functional Flexibility in Generative AI Interfaces: Text Editing with LLMs through Conversations, Toolbars, and Prompts. *arXiv preprint arXiv:2410.10644* (2024). <https://arxiv.org/abs/2410.10644>